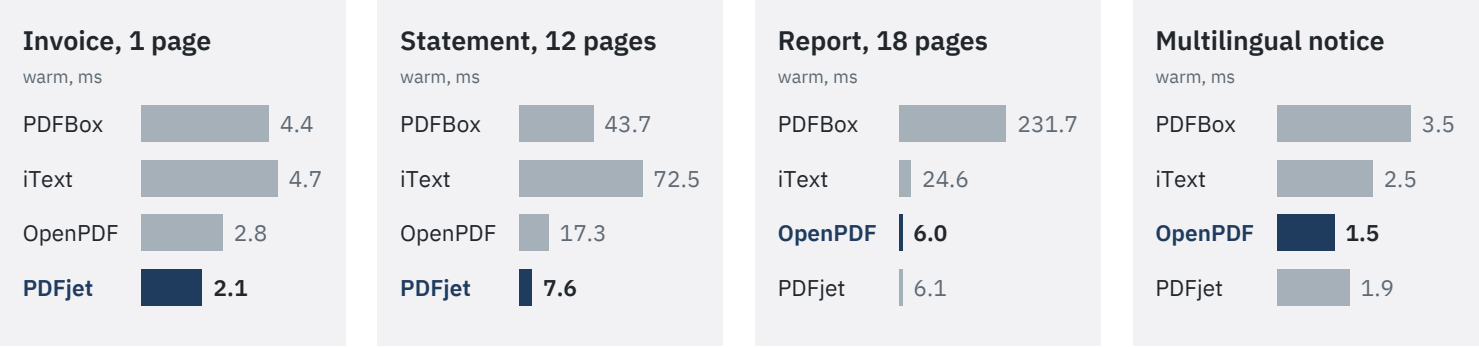
Generating PDFs in Java: four libraries compared

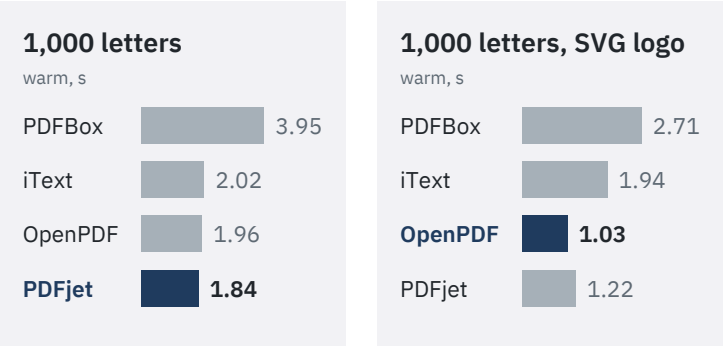
Five business documents and a batch of letters, made by each library the way its own documentation shows, every PDF checked, in **IBM Plex Sans**, and the invoice, the report and the notice again in **Noto Sans**. Written by the author of PDFjet, with everything needed to run it again published; the other measurements, the features of each library, the method and the tables of every figure are on the pages after this one.

Lower is better, and the best is in navy.

Time per document. These tests use IBM Plex Sans.



The 1,000 letters, with the logo as PNG and as SVG. These tests use IBM Plex Sans.



The libraries

Apache PDFBox 3.0.8: Apache License 2.0, Java 8 and later.

OpenPDF 3.0.5: LGPL-2.1 or MPL-2.0, Java 21 and later.

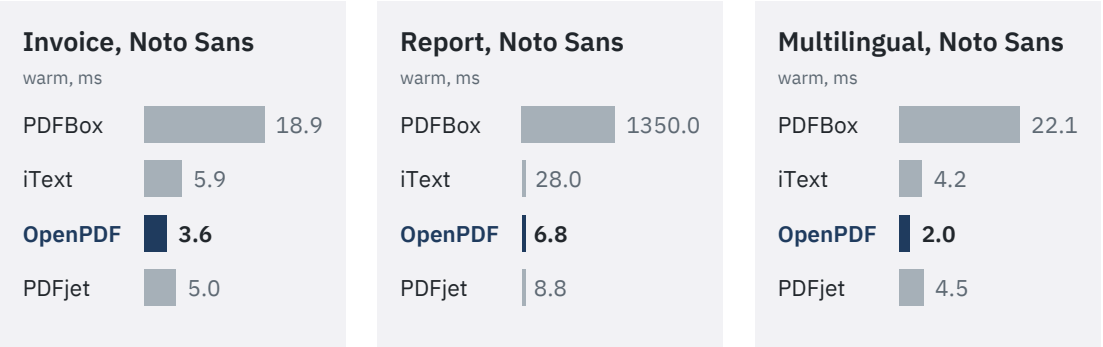
iText Core 9.8.0: AGPL-3.0 or a commercial license, Java 8 and later.

PDFjet 9.1.5: MIT License, Java 8 and later; also in C#, Go and Swift.

With Noto Sans, a large font

Lower is better, and the best is in navy.

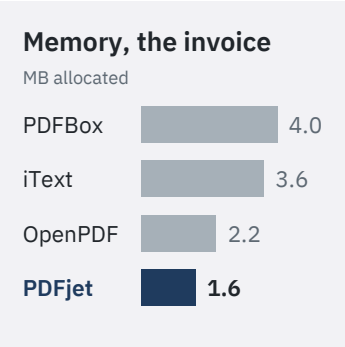
Time per document, Noto Sans. These tests use Noto Sans, which has many more glyphs than IBM Plex Sans.

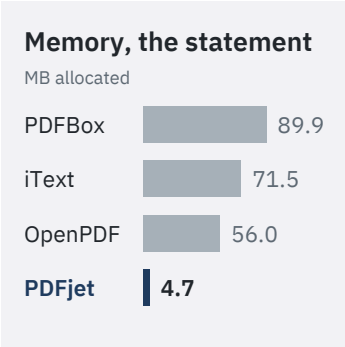


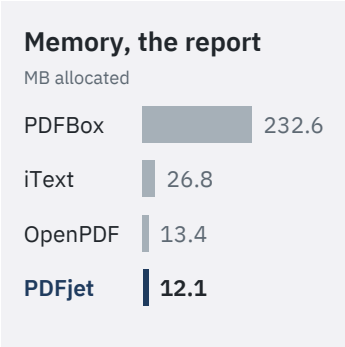
The memory each document allocates

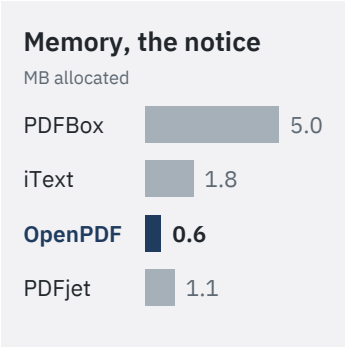
Lower is better, and the best is in navy.

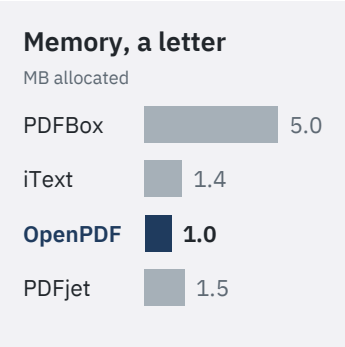
Memory allocated per document. These tests use IBM Plex Sans.







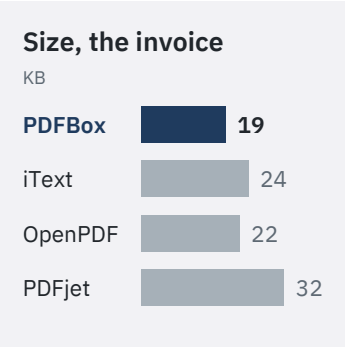


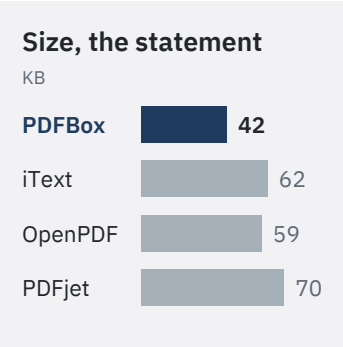


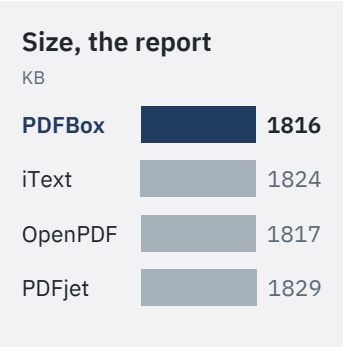
The size of the PDFs

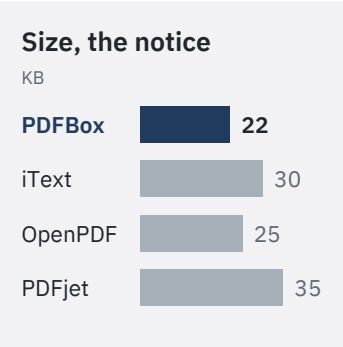
Lower is better, and the best is in navy.

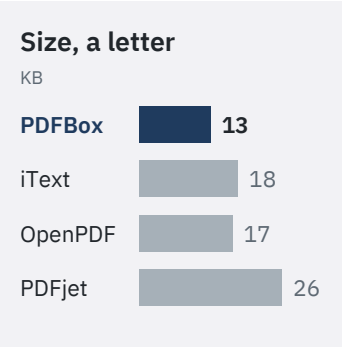
The size of the PDFs. These tests use IBM Plex Sans.







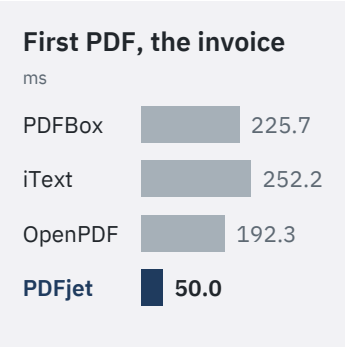


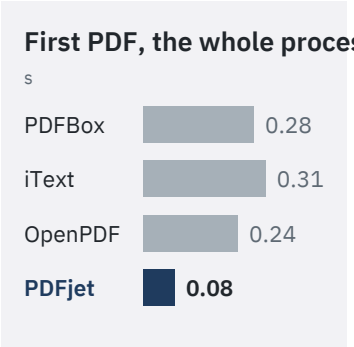


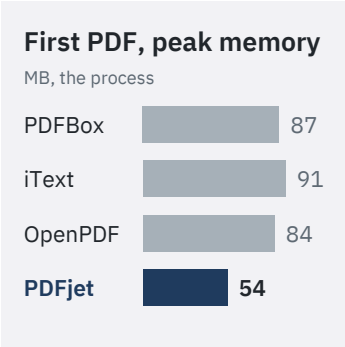
The first PDF of a process

Lower is better, and the best is in navy.

The first PDF of a fresh process. These tests use IBM Plex Sans.



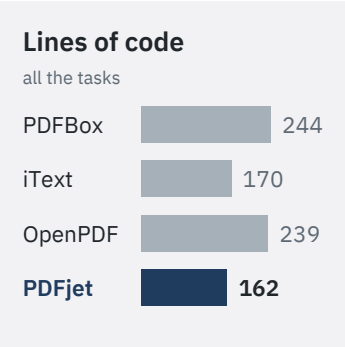




The lines of code of the programs

Lower is better, and the best is in navy.

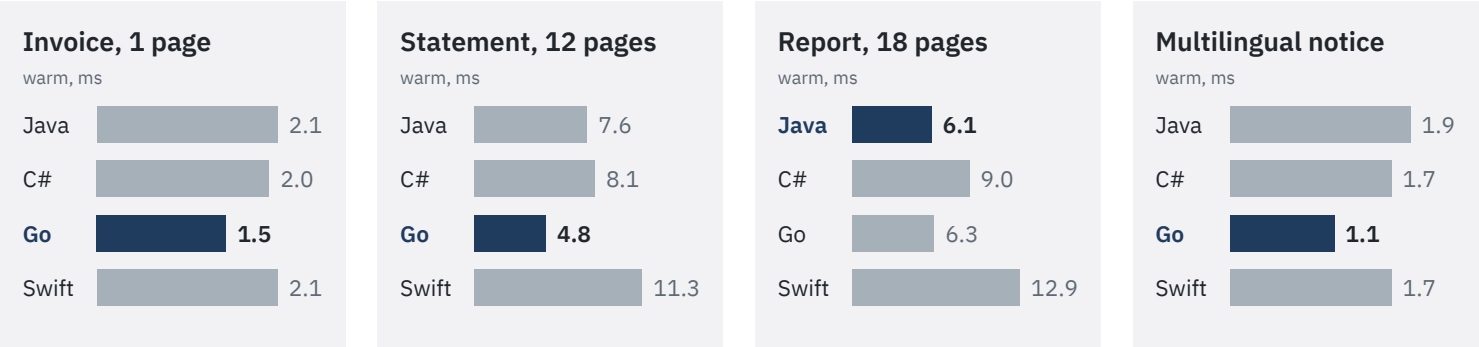
Lines of code, all the tasks. Without blank lines and comments; the letters with an SVG logo are not counted.



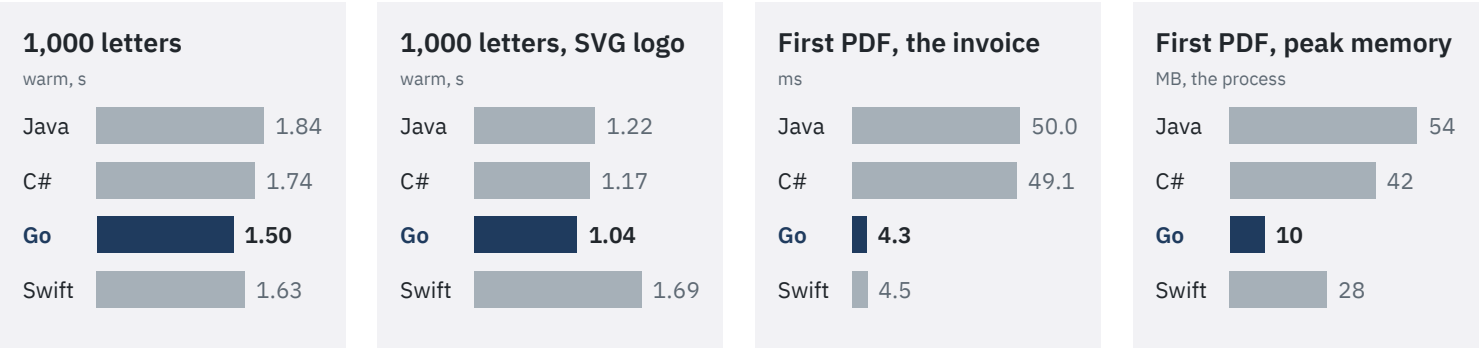
PDFjet in Java, C#, Go and Swift

Lower is better, and the best is in navy.

Time per document, the four ports. These tests use IBM Plex Sans.



The letters and the first PDF, the four ports. These tests use IBM Plex Sans.



I wrote PDFjet, one of the four libraries compared here, and have been writing it since 2001. For the last two months I have worked on it with Claude, an AI coding assistant. I commissioned this benchmark; Claude chose the documents, their content and the way each library is used, wrote the programs, ran them and wrote the tables. Both are reasons to be suspicious of the results, so everything needed to check them is published in github.com/edragoev1/pdf-benchmarks: the specification of the documents, every program, the scripts that run and check them, and the raw results.

If we have configured a library badly, or used it in a way its users would not, please tell us or open an issue, and we will fix the program and run the benchmark again.

The libraries

Library	Version	License	Java
PDFjet (core)	9.1.5 (main at 84e73b5f until the release)	MIT	8 and later
Apache PDFBox	3.0.8	Apache 2.0	8 and later
iText Core	9.8.0	AGPL-3.0 or commercial	8 and later
OpenPDF	3.0.5	LGPL-2.1 or MPL-2.0	21 and later

The newest releases on Maven Central on 26 October 2026. For the letters with an SVG logo, the modules the others need to draw SVG, each at its newest release: iText's svg module 9.8.0, Apache Batik 1.20, and for PDFBox the pdfbox-graphics2d bridge 3.0.5.

Each library is used the way its own documentation shows: iText's and OpenPDF's `Document` for flowing text, PDFBox's content streams with lines wrapped by hand, as it has no layout of its own, and PDFjet's TextFrame and Table, and for the report TextFlow. Fonts are kept the way a server would keep them: iText and OpenPDF with their own default caches, PDFBox with its TrueType font parsed once and reused, as its API allows; PDFjet has no font cache and reads its fonts for each document.

Features

What each library has, as of the versions above, from its source at the release tag, its Maven POM and its documentation; only the features the documents use were tested here. A module is another jar of the same project; an add-on is a separate product of the same company.

PDFBox has no table of its own, so its programs here draw the tables by hand: a gray box behind the header, each cell's text measured to align the numbers right, a rule under each row, the header again on each new page, and the page numbers in a second pass once the pages are counted.

The other jars each of the others needs at run time, from its POM: PDFBox pdfbox-io, fontbox and commons-logging; iText io,

commons, jackson-databind, slf4j-api and bouncy-castle-connector, and its layout jar for tables and flowing text; OpenPDF brotli4j with a native jar for the platform. pdfCalligraph, pdfRender, pdfHTML, pdfSweep and pdfOCR are iText add-ons.

Far more people have worked on PDFBox, iText and OpenPDF than on PDFjet, and far more people use them, with more answers on the web for every question; iText also has a company behind it with paid support.

Feature	PDFBox	iText	OpenPDF	PDFjet
Tables	no	✓	✓	✓
Text wrapped into paragraphs	no, by hand	✓	✓	✓
Text flowing onto new pages	no	✓	✓	✓
Tagged PDF, PDF/UA-1	tags by hand	✓	tags by hand	✓
PDF/UA-2	no	✓	no	no
PDF/A	metadata by hand	1a to 4f	1a to 3u	1a to 3b
SVG	no	svg module	no	✓
Barcodes, 1D	no	✓	✓	✓
QR codes	no	✓	no	✓
Charts	no	no	no	bar, line, scatter, donut
Markdown to PDF	no	no	no	✓
HTML to PDF	no	pdfHTML add-on	openpdf-html module	no
Images	JPEG, PNG, GIF, BMP, TIFF	those, WMF, JPEG 2000, JBIG2, WebP	those, WMF, JPEG 2000	JPEG, PNG, BMP
Kerning	no	pdfCalligraph, commercial	through Java AWT or Apache FOP	✓
Arabic	no	pdfCalligraph, commercial	✓	✓
Devanagari and other Indic scripts	some	pdfCalligraph, commercial	through Java AWT or Apache FOP	no
Hyphenation	no	hyph module	✓ patterns not included	soft hyphens only
Extract text	✓	✓	✓	no

Feature	PDFBox	iText	OpenPDF	PDFjet
Render a page to an image	✓	pdfRender, commercial	openpdf-renderer module	no
Change existing PDFs	✓	✓	✓	add content, merge, split
Fill or create form fields	✓	✓	✓	no
Digital signatures	✓ with Bouncy Castle	sign module, with Bouncy Castle	✓ with Bouncy Castle	no
Password encryption, AES-256	✓	✓	✓	✓
Certificate encryption	✓ with Bouncy Castle	✓ with Bouncy Castle	✓ with Bouncy Castle	no
PDF 2.0	no option found	✓	a 2.0 header by default	no, PDF 1.7
Object streams, for smaller files	✓ by default	✓ an option	✓ an option	no
Redaction	no	pdfSweep add-on	no	no
OCR	no	pdfOCR add-on	no	no
XFA forms	reads the data	fills the data; flattens, commercial	reads, fills the data	no
No other jars needed	no	no	no	✓
Java	8 and later	8 and later	21 and later	8 and later
Also for	—	.NET	—	C#, Go, Swift
License	Apache 2.0	AGPL-3.0 or commercial	LGPL-2.1 or MPL-2.0	MIT

What was measured

Six tasks, specified exactly in SPEC.md, the same text and the same layout in every library:

- **invoice**: 1 page, the PDFjet logo, two address blocks, a table of 15 rows, totals.
- **statement**: 12 pages, a table of 600 transactions with running footers "Page n of N".
- **report**: 18 pages, 120 paragraphs of flowing text and 4 photos, JPEGs of 1870 × 2420 pixels, 1.8 MB in all.
- **multilingual**: 1 page, a notice in eight languages: English, French, German, Polish, Turkish, Greek, Bulgarian and Ukrainian.

- **letters**: 1,000 one-page letters with the logo, a PNG, timed as the batch.
- **letters with an SVG logo**: the same letters with the same logo as SVG.

All in **IBM Plex Sans**, embedded as subsets; the invoice, the report and the notice again in **Noto Sans**, a much larger font.

Method. Each library in its own fresh JVM, in a random order, three rounds. Each task is warmed up for at least 20 runs and 3 seconds, then timed over 15 runs (the letters: 2 warm-up batches, 5 timed); the figure is the median of the three round medians. An asterisk (*) marks a figure whose rounds differ by more than 5%. The first PDF of a fresh process is the median of 5 processes. OpenJDK 25, AMD Ryzen 5 5600G, Linux, nothing else running. After the run every PDF is checked, and the log is published: MuPDF opens it with no warning, it has the expected pages, the check strings of SPEC.md are in its text (the last sentence of every language for the notice), both fonts are embedded as subsets, and qpdf --check passes.

Results

Every measurement is drawn as a chart on the first pages, and the tables of every figure, with PDFjet in C#, Go and Swift beside the four libraries, are in the appendix at the end.

To draw the SVG logo the others need another module or library, and each was given the one its users would choose:

Library	to draw SVG it needs
PDFBox	Apache Batik and pdfbox-graphics2d
iText	iText's svg module
OpenPDF	Apache Batik
PDFjet	nothing else

PDFjet in C#, Go and Swift

PDFjet is also written in C#, Go and Swift: the same classes and the same public API in each, and the same documents laid out the same way, which an automated check compares on every change. The last three columns of the tables in the appendix are the same documents made by each of them. For a program that makes PDFs, what counts is what the port of its language makes, and how fast: that is what those columns show.

A port can use only what its language offers, and Deflate shows it most. Every PDF compresses its pages, and each language offers its own Deflate:

Port	Deflate	Level
Java	zlib (java.util.zip), native C	5
C#	zlib (ZLibStream), native C	6, .NET's Optimal
Go	compress/zlib, written in Go	6
Swift	PDFjet's own, written in Swift	its own, searching less than zlib

Java offers zlib at every level, and PDFjet uses 5, the fastest for the size it gives. .NET offers the same zlib at three levels only, Fastest, Optimal and SmallestSize, with no 5 between them. Go offers its own Deflate, written in Go; native zlib would need cgo, which a library in pure Go avoids. Swift has no Deflate of its own on every platform PDFjet runs on, so the Swift port has its own encoder, which searches less: its PDFs are the largest of the four, the statement 79 KB to the others' 70.

The ports are not Java in other syntax. Each is written for its language, with its own standard library and idioms, and the results show it: one design translated would rank the same in every document, and these do not. If you find an omission that puts your favorite language at a disadvantage, a slow way of doing something where the language has a faster one, please tell us: we will be happy to adopt it in the library and run the test again.

Reproduce it

Everything is in github.com/edragoev1/pdf-benchmarks: SPEC.md, the text of the notice, the logo and the photos, the program of each library and port, the run and check scripts, and the raw results with their logs. java/run.sh runs the four Java libraries (about 30 minutes) and java/check.sh checks every PDF; dotnet/, go/ and swift/ run PDFjet's other ports the same way; ports-nocompress.sh runs the four ports with their pages not compressed; make-tables.py makes every table of this report from the results.

Judgment calls

- The table rows of every library are 13.2 pt high, PDFjet's default for IBM Plex Sans at 9 pt; iText's and OpenPDF's cell padding was adjusted to match it, as without it their page counts differ.
- OpenPDF uses `setStrictImageSequence(true)`, or it moves text above a photo that goes to the next page.
- iText and OpenPDF use their large-table modes for the statement, as their documentation shows for long tables. iText's ordinary Table was timed too, and was slower (about 106 ms against 72).
- PDFBox draws one rule under each table row and one header rectangle, where the others draw a border under each cell: slightly less work for PDFBox.
- PDFBox applies the font's ligatures (GSUB) to its text, which the others do not; with Noto Sans, whose tables are large, this is the likely cause of its slow report, which allocates 2.8 GB, though it was not profiled.

- PDFjet's Noto report is 21 pages to the others' 20: it spaces lines by the font's own height.

The ports, looked into

For the curious, not for choosing a port: where the time of the 12-page statement goes, from a profile of each port at its steady state (perf, a long run so that the JIT runtimes have finished compiling):

Share of the time	Java	C#	Go	Swift
compressing	59%	43%	32%	16%
the runtime: GC, reference counting, allocating, copying	13%	18%	26%	30%
strings, the language's own	—	—	—	14%
PDFjet's own code, and the rest	28%	39%	42%	40%

(Measured 26 October with every port at level 6, before Java went to level 5; to be measured again with 9.1.5.)

C# spends about a tenth of its report copying and zeroing memory buffers for the photos, where Java spends a twentieth, which is part of the gap between them in that document.

The same documents with their pages not compressed, `setCompression(false)`, show each port's code without its compressor; the fonts and the photos are compressed as before. Three rounds in a random order, 20 warm-ups and 15 timed runs, as in the main run, by `ports-nocompress.sh`, ms:

Document	Java (OpenJDK 25)	C# (.NET)	Go	Swift
invoice, compressed	1.9	1.9	**1.6 ***	2.3 *
invoice, not compressed	1.8	1.9 *	**1.6 ***	1.8 *
statement, compressed	5.7	7.9	**4.9**	11.3 *
statement, not compressed	**2.7**	4.5 *	3.5 *	8.4
report, compressed	**5.9**	8.8 *	6.4 *	13.1
report, not compressed	**4.1**	7.6 *	4.9	12.2 *

Appendix: the tables

Time per document, IBM Plex Sans

Time per document, warm, ms (the letters: the batch of 1,000, s). In every table lower is better, faster, less memory, smaller files, fewer lines, and the best of the four Java libraries is in bold. The last three columns are PDFjet in C#, Go and Swift, shown with them but not ranked against them.

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice, 1 page	4.4 *	4.7 *	2.8 *	**2.1**	2.0	1.5 *	2.1
statement, 12 pages	43.7	72.5 *	17.3	**7.6**	8.1	4.8	11.3
report, 18 pages	231.7	24.6 *	**6.0**	6.1 *	9.0	6.3	12.9
multilingual, 1 page	3.5	2.5	**1.5**	1.9	1.7	1.1 *	1.7
1,000 letters, s	3.95	2.02	1.96	**1.84**	1.74	1.50	1.63
1,000 letters, SVG logo, s	2.71	1.94	**1.03**	1.22	1.17	1.04	1.69

Time per document, Noto Sans

With Noto Sans, a large font, ms:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	18.9 *	5.9 *	**3.6 ***	5.0	4.7	2.7 *	4.4
report	1,350	28.0	**6.8 ***	8.8	12.0 *	7.6	15.1
multilingual	22.1	4.2	**2.0**	4.5	4.4	2.4 *	2.9

The first PDF of a process

The first PDF of a fresh process, the invoice, as a command-line tool, a serverless function or a server just started pays it. The peak memory is that of the whole process, mostly the JVM itself:

	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
the invoice, ms	225.7	252.2	192.3	**50.0**	49.1	4.3	4.5
the whole process, s	0.28	0.31	0.24	**0.08**	0.10	0.00	0.01

	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
its peak resident memory, MB	87	91	84	**54**	42	10	28

Memory allocated per document

Memory allocated per document, MB:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	4.0	3.6	2.2	**1.6**	1.5	2.7	—
statement	89.9	71.5	56.0	**4.7**	5.5	4.8	—
report	232.6	26.8	13.4	**12.1**	12.6	9.1	—
multilingual	5.0	1.8	**0.6**	1.1	1.1	1.3	—
a letter	5.0	1.4	**1.0**	1.5	1.4	2.2	—

The size of the PDFs

The size of the PDFs, KB:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	**19**	24	22	32	32	32	34
statement	**42**	62	59	70	70	70	79
report	**1,816**	1,824	1,817	1,829	1,829	1,830	1,833
multilingual	**22**	30	25	35	35	36	38
a letter	**13**	18	17	26	26	27	28

Lines of code

Lines of code of each program, without blank lines and comments; "helpers" are the methods a library's tasks share (fonts, cells, footers, word wrapping), counted in the total. The letters with an SVG logo are not in the total:

	invoice	statement	report	letters	multilingual	helpers	total
PDFBox	**28**	26	53	23	24	90	244
iText	39	30	**22**	**17**	**17**	45	170

	invoice	statement	report	letters	multilingual	helpers	total
OpenPDF	57	37	32	25	23	65	239
PDFjet Java	31	**25**	32	20	**17**	**37**	**162**
PDFjet C#	32	25	32	20	17	35	161
PDFjet Go	36	26	35	23	20	35	175
PDFjet Swift	29	22	31	18	15	45	160